

JavaScript Regular Expressions Cheat Sheet

A comprehensive guide to JavaScript regular expressions (regex) for pattern matching, text manipulation, best practices, and real-world tips.

Tip

Complete Dev Roadmap with SaaS Boilerplate at thedevspace.io/

Regex Syntax & Flags

```
1 // basic literal with ignore-case flag
2 const re = /abc/i;
3 const str = "ABC";
4 re.test(str); // true
```

- Use `/pattern/flags` for inline regex literals
- Common flags: `g` (global), `i` (ignore case), `m` (multiline), `u` (unicode), `s` (dotAll)
- Use the `RegExp` constructor when building patterns dynamically

Character Classes & Quantifiers

```
1 // match 2 to 4 digits globally
2 const re = /\d{2,4}/g;
3 "Year: 2023".match(re); // ['2023']
```

- `\d` digit, `\w` word, `\s` whitespace, `.` any character
- Quantifiers: `*` (0+), `+` (1+), `?` (0 or 1), `{n,m}` (n to m)
- Use non greedy quantifiers (`*?` , `+?`) to avoid overmatching

Anchors & Boundaries

```
1 /^Hello/.test("Hello world"); // true
2 /world$/.test("Hello world"); // true
3 "cat dog".match(/\bcat\b/); // ['cat']
```

- **^** matches start, **\$** matches end of string
- **\b** marks a word boundary, useful for whole-word matches
- Anchors help avoid accidental partial matches

Groups & Alternation

```
1 const re = /(cat|dog)/;  
2 re.test("my dog"); // true  
3 const re2 = /(?:foo|bar)/; // non-capturing
```

- Use parentheses for grouping and alternation with `|`
- Use `(?:...)` to group without capturing when you only need grouping
- Capture groups are useful but avoid unnecessary captures for performance

Common Patterns

```
1 // common pattern examples
2 const email = /^[^@\s]+@[^\s]+\.[^\s]+$/;
3 const url = /https?:\/\/\./+;/
4 const phone = /\d{3}-\d{3}-\d{4}/;
5 const hex = /[0-9a-fA-F]{6}/;
6 const strongPass = /^(?=.*[A-Z])(?=.*\d){8,}$/;
```

- Keep patterns as simple as possible while meeting requirements
- Validate with tests and sample inputs to avoid false positives
- For complex validation prefer dedicated parsers or libraries

Replace & Extract

```
1 // replace digits with dash and extract matches
2 const text = "foo123bar";
3 text.replace(/\d+/g, "-"); // 'foo-bar'
4 [...text.matchAll(/(\d+)/g)]; // iterator of groups
```

- Use `String.replace` for substitutions and `String.match` / `matchAll` for extraction
- `matchAll` returns an iterator of match objects including groups (ES2020+)
- Prefer global flags for repeated matches and iterate results safely

Escaping & Special Characters

```
1 // escape special characters before building a
   dynamic regex
2 function escapeRegex(s) {
3   return s.replace(/[\.\*\+\?\^\$\{\}\(\)|\[\]\\"/>

```

- Escape `. * + ? ^ $ { } ( ) | [ ] \` when including literal characters
- Use `String.raw` for clearer literal patterns in templates
- When constructing from user input always escape to avoid injection-like surprises